1. Strong-rho assurance round (2026-10-16)

Study: `research/rho_assurance_20261016/` · **Protocol:** `PROTOCOL.md`, declared before the evidence; each Q4 addendum was written before its code · **Baseline:** `d5382a530` · **Candidate:** `448be6146` · **Host:** one x86-64 cloud VM (Intel Xeon @ 2.10 GHz, AVX-512, 4 logical CPUs). It exposes no hardware PMU, so there are no cycle or IPC counts.

The strong rho (`src/cryptanalysis/koblitz_strong_rho.rs`) is the reference every Koblitz `vs_rho` comparison prices index calculus against. This round asks four questions about it. The labels are **measured** (a run record), **derived** (computed from measured values, as shown) and **proved** (machine-checked).

1.1. Q1. Is it correct?

- **Lean 4 (proved).** 34 theorems in core Lean v4.34.1, no Mathlib, no `sorry`, axioms `propext`, `Classical.choice` and `Quot.sound` only:
  - the least-rotation lemma behind `NormalBasis::least_rotation`, with the Rust candidate mask shown exact;
  - Montgomery batch inversion, with zero entries;
  - preservation of $P = aG + bQ$ by starts, jumps, canonicalization and escapes, and the collision formula.
  - Gaps: the curve group is modelled as the scalar field; the field arithmetic and basis maps are not verified.
- **TLA+ / TLC (exhaustive on small instances).** A model of `solve` over $\mathbb{Z}_R$ (R = 31, 61, 113; 1–3 lanes; both fruitless modes). All 41 configs meet their declared verdicts:
  - 18 models hold `RepInv`, `CanonInv`, `NoFailedCollision` and `SolvedCorrect`;
  - 2 liveness configs terminate;
  - 7 mutants fail;
  - 12 witnesses reach every branch.
- **Rust (measured).**
  - An exhaustive least-rotation test over every word up to 16 bits.
  - Debug-build checks of $P = aG + bQ$ inside `solve`.
  - All 12 strong-rho tests pass.
- **Findings.** None changes an answer.
  - Infinity doubles as the `previous` sentinel.
  - Escape steps skip the walk cap; escape is off in the reference.
  - The cycle-length doc said ≤ 4 where the check catches 1–5. Fixed.
  - Merged walks can escape apart.

1.2. Q2. Does it take the steps it should? (measured, counted units)

400 fixtures per size. The ratio is walk steps $/\sqrt{\pi r/2A}$.

curve	mean	95% CI	sd	with starts
`icv1-f2m41-tm2308219-7f48b14a`	0.994	0.944–1.044	0.513	1.061
`icv1-f2m53-tm56619371-dac20a85`	0.922	0.873–0.971	0.504	0.984
`icv1-f2m61-t158598901-ab42b6c5`	0.943	0.894–0.993	0.500	1.007

Every upper end is below the declared 1.10 limit: the walk is not inflated.

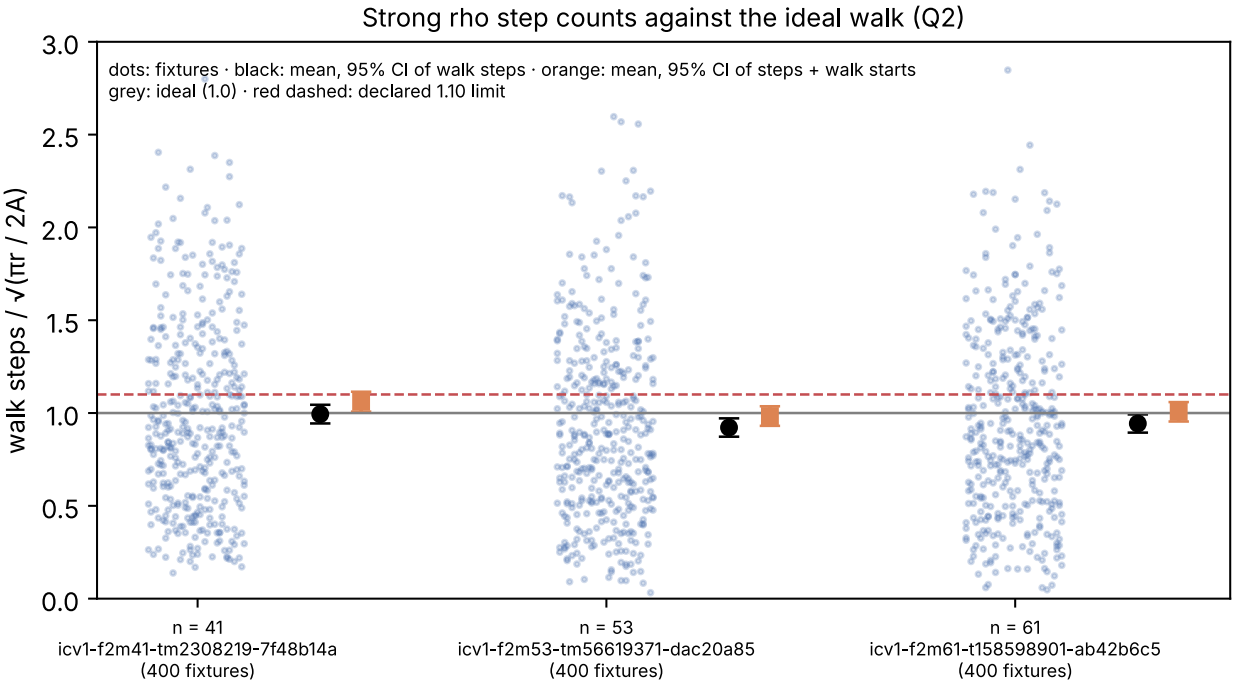


Figure 1: Q2: steps against the ideal, 400 fixtures per size (`runs/q2/`).

1.3. Q3. What does a step cost? (measured, Callgrind, n = 53, 4 fixtures)

- **Per step:** 1,196 instructions.
- **Per field operation:** `Gf2::mul` and `Gf2::sqr` are 21 instructions each (`examples/gf2_mul_cost.rs`, difference of two runs).
- **In M units (derived):** about 57 M per step, against an affine batched floor of about $5M + 1S + I/32 \approx 6$–7 M.
- **Where it went:**
  - canonicalization was 52% of instructions, mostly four byte-table basis conversions at about 14 instructions per lookup;
  - SipHash in the distinguished-point table was about 6%.

1.4. Q4. A faster reference with identical walks (measured)

- **The changes:**
  - `Linear::apply2` converts both coordinates in one pass;
  - `add_mod` drops a 128-bit `%`;
  - the table hashes with a fixed mixer.
- **Identity:** every fixture line at n = 53 and 61 is identical, timing aside.
- **Instructions:** 3,628,440,819 ⟶ 2,847,167,115 (−21.5%), or 1,196 ⟶ 938 per step.

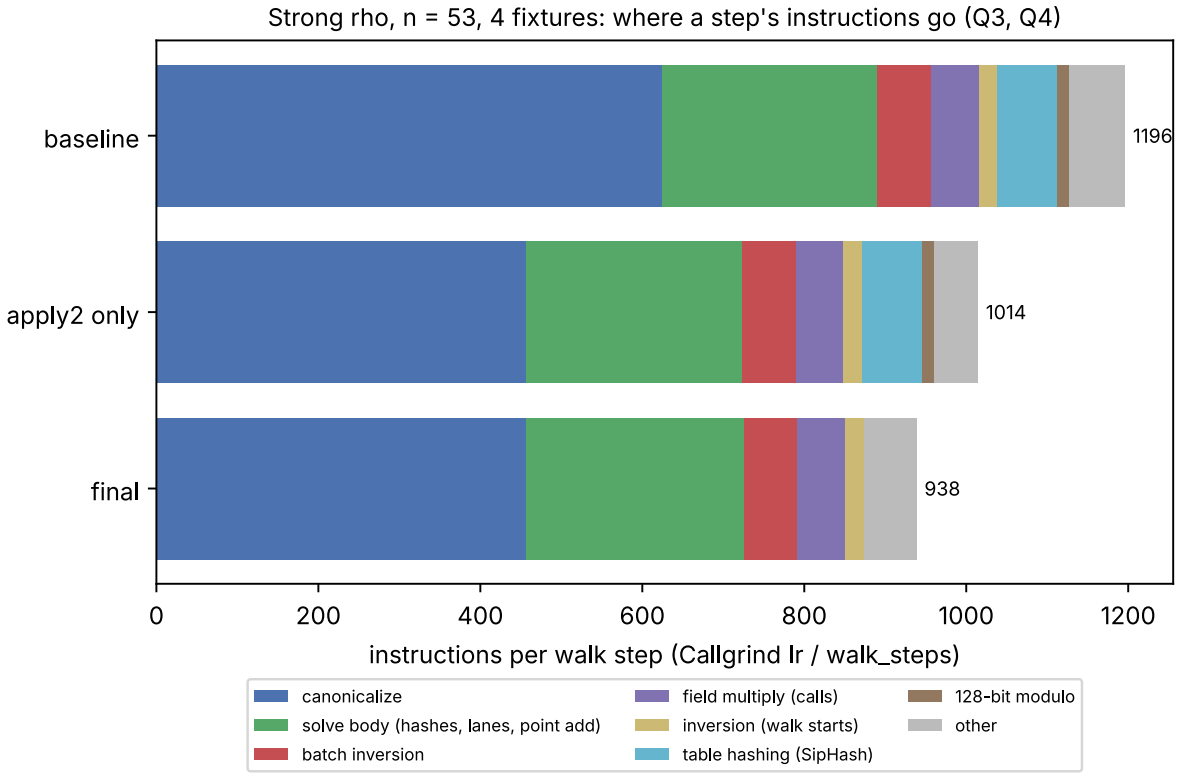


Figure 2: Q4: instructions per walk step by function (`runs/q4/callgrind/`).

Native time. 32 processes under `isolated_bench`; 2 were contended and are excluded. On this VM the A/A spreads are wide (11–18%), so the instruction count is the result.

curve	baseline ns/step	candidate ns/step	ratio	A/A spread
`icv1-f2m53-…`	144.4	141.2	1.023	10.7% (not established)
`icv1-f2m61-…`	168.4	139.2	1.209	18.3% (barely outside)

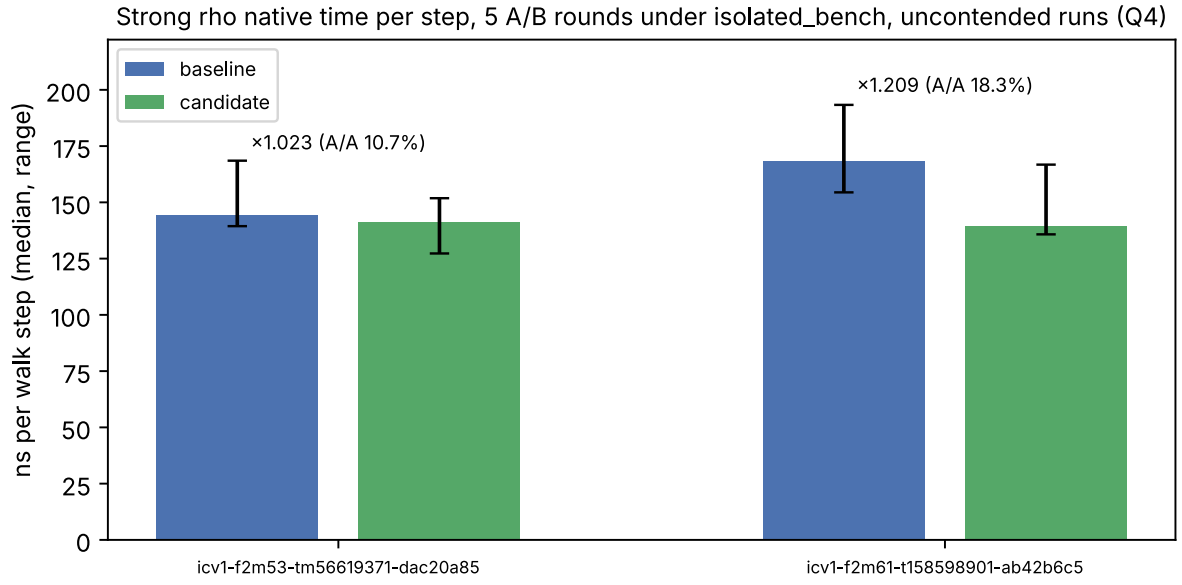


Figure 3: Q4: native ns per walk step, uncontended runs (`runs/q4/time/`).

1.5. Scope and consequences

- **Counted units.** GAE records do not change.

- **Wall time and instructions.** Future IC/rho comparisons against this reference price rho about 22% cheaper per step in instructions, which makes them harder for index calculus. Earlier rows are not re-measured.
- **What remains.** The reference still spends about 45 M per step against a floor near 7 M. A per-step comparison is still generous to index calculus, by an unmeasured amount.
- **What is not shown.**
  - TLC checks small instances of a model.
  - Lean abstracts the curve and the field.
  - Neither is a refinement proof of the Rust.
  - Only one host was measured, with no hardware counters.